SOLID Principles — Complete Evolution (SRP, OCP, LSP, ISP, DIP + Strategy + DI)

🐦 The Problem: Design a Bird

Requirement

Design a `Bird` class with attributes and behaviors like eat, fly, and make sound.

❌ Version 0 — Naive Design (if-else)

```
class Bird {
    String type;

    void makeSound() {
        if (type == "sparrow") System.out.println("chi chi");
        else if (type == "pigeon") System.out.println("gutargoon");
        else if (type == "crow") System.out.println("kaw kaw");
    }
}
```

Problems

- ❌ Readability (huge if-else chains)
- ❌ Testability (hundreds of test cases)
- ❌ Parallel dev issues (merge conflicts)
- ❌ SRP violation (multiple reasons to change)

⚙️ Version 1 — SRP Improvement (Delegation)

```
void makeSound() {
    if (type == "sparrow") makeSparrowSound();
    if (type == "pigeon") makePigeonSound();
}
```

Outcome

- ✅ Better SRP
- ❌ Still violates OCP (modifying code for new birds)

🧱 Version 2 — OOP Design (SRP + OCP)

```
abstract class Bird {
    abstract void eat();
    abstract void fly();
    abstract void makeSound();
}

class Sparrow extends Bird { ... }
class Pigeon extends Bird { ... }
```

Outcome

- ✅ No modification of existing code
- ✅ Each class has one responsibility

⚠️ LSP Problem

Some birds (like Penguin) cannot fly → violates expectation.

Fix

Split hierarchy:

- FlyingBird
- NonFlyingBird

❌ Version 2.3 — Class Explosion

With features like flying/talking:

- 2 features → 4 classes
- n features → 2^n classes ❌

🔌 Version 3 — Interfaces (ISP Applied)

```
abstract class Bird {
    abstract void eat();
}

interface Flyable { void fly(); }
interface Talkable { void makeSound(); }
```

Key Idea

- Capabilities are **opt-in**
- Birds implement only what they can do

Benefit

- ✅ No class explosion → n interfaces instead of 2^n classes

💃 Version 3.1 — ISP (Interface Segregation)

New requirement: Birds can dance

❌ Wrong: Add `dance()` to Flyable ➡️ Violates ISP

✅ Correct:

```
interface Danceable { void dance(); }
```

Insight

👉 ISP = SRP for interfaces

🧩 Version 4 — Clean Capability Design

- Birds extend `Bird`
- Implement only required interfaces:

Example:

- Sparrow → Flyable, Talkable, Danceable
- Penguin → Talkable, Danceable

Outcome

- ✅ Flexible
- ✅ Extensible
- ✅ OCP + ISP satisfied

🔁 Problem — DRY Violation in fly()

- Sparrow, Peacock → same (slow fly)
- Crow, Pigeon → same (fast fly)

❌ Duplicate logic

🚀 Version 5 — Strategy Pattern + DIP

Step 1: Abstraction

```
interface Flyator {
    void flightAlgorithm();
}
```

Step 2: Implementations

```
class FastFlyator implements Flyator { ... }
class SlowFlyator implements Flyator { ... }
```

Step 3: Composition in Bird

```
class Sparrow implements Flyable {
    private Flyator ref;

    public void fly() {
        ref.flightAlgorithm();
    }
}
```

🔌 Plug & Socket Model (IoC)

```
sparrow.plugFlightAlgo(new SlowFlyator());
```

- Sparrow = Socket
- Flyator = Interface
- Concrete classes = Plugs

Outcome

- ✅ Loose coupling
- ✅ Runtime flexibility

📌 Dependency Inversion Principle (DIP)

Definition:

> High-level modules should not depend on low-level modules. Both should depend on abstractions.

Meaning

- Depend on interface (`Flyator`)
- Not on implementation (`SlowFlyator`)

🔄 Inversion of Control (IoC)

- Before: Object creates dependency ❌
- After: Client injects dependency ✅

💉 Dependency Injection (DI)

1. Constructor Injection

```
class Sparrow {
    public Sparrow(Flyator ref) {
        this.ref = ref;
    }
}
```

- ✅ Always valid object
- ❌ Cannot switch behavior easily

---

2. Method Injection

```
sparrow.plugFlightAlgo(new SlowFlyator());
```

- ✅ Can switch behavior anytime
- ❌ Object may be incomplete

⚖️ Best Practice

👉 Use both:

- Constructor → mandatory dependencies
- Method → dynamic behavior

🚫 Field Injection

- ❌ Hidden dependencies
- ❌ Hard to test
- ❌ Avoid

🧠 Final Evolution Flow

```
V0 → if-else chaos
V1 → delegation (SRP)
V2 → inheritance (OCP)
V3 → interfaces (ISP)
V4 → clean capability design
V5 → strategy + DIP + DI
```

🏁 Final Takeaways

- SRP → one responsibility
- OCP → extend, don't modify
- LSP → don't break expectations
- ISP → small interfaces
- DIP → depend on abstraction

🔥 Interview One-Liner

👉 "We decouple behavior using interfaces and strategy pattern, and inject dependencies from outside to achieve loose coupling, flexibility, and scalability."
